SOLAR▸SWARM

Bundle

Martin Pham | Perth, AU

Solar Swarm — Master Documentation Index

Solar Swarm is a two-system platform built to generate recurring revenue from Australian solar SMEs using AI automation.

Owner: Martin Pham | Perth, Australia | AI Automation Consultant

Target: $1,500–2,000 AUD/month retainer per solar client

Margin: 80–90% (total costs ~$400 AUD/month)

What the System Does

System 1 — Autonomous Agent Swarm

A 3-tier hierarchy of AI agents that continuously generates, evaluates, and runs business experiments. The General (Tier 1) creates experiment ideas via GPT-4o, scores them using a 4-component confidence system, runs them through a red team critic, then routes them: auto-proceed, human gate, or auto-kill. Budget is allocated using a 25% Fractional Kelly Criterion. Agents communicate via a SQLite-backed message bus. Results feed back as pheromone signals that bias future experiment generation.

System 2 — Solar Sales Automation

A production CRM automation layer targeting Australian solar companies with 5–15 salespeople. When a lead arrives via GoHighLevel webhook, the system qualifies them with GPT-4o (score 1–10), triggers outbound AI voice calls for hot leads via Retell AI, updates the GHL pipeline, sends Slack alerts, and produces weekly performance reports.

ASCII Architecture Diagram

```
┌──────────────────────────────────────────────────────────────────────┐
│                          SOLAR SWARM                                 │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────────┐    │
│  │                 SYSTEM 1 — AGENT SWARM                       │    │
│  │                                                              │    │
│  │  TIER 1       ┌──────────────────┐                           │    │
│  │               │   master_agent   │  (The General)            │    │
│  │               │   every 6h       │                           │    │
│  │               └──────────┬───────┘                           │    │
│  │                          │ generates + routes experiments    │    │
│  │  TIER 2       ┌──────────┴──────────────────────┐            │    │
│  │               │  research_agent │ content_agent  │           │    │
│  │               │  analytics_agent                 │           │    │
│  │               └──────────┬──────────────────────┘            │    │
│  │                          │ spawns workers                    │    │
│  │  TIER 3       ┌──────────┴──────────────────────┐            │    │
│  │               │ qualification │ proposal │ solar │           │    │
│  │               │ research │ report │ red_team      │           │    │
│  │               └──────────────────────────────────┘            │    │
│  │                                                              │    │
│  │  SUPPORT     scout_agent (daily 08:00 UTC)                   │    │
│  │              mutation_engine (Monday 22:30 UTC)              │    │
│  │              ab_tester (daily 10:00 UTC)                     │    │
│  └──────────────────────────────────────────────────────────────┘    │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────────┐    │
│  │              SYSTEM 2 — SOLAR SALES AUTOMATION               │    │
│  │                                                              │    │
│  │  GHL Webhook → qualify lead → score 1–10 → route             │    │
│  │  score ≥ 7 → Retell outbound call + Slack alert              │    │
│  │  post-call → GPT extract → GHL update → task                 │    │
│  └──────────────────────────────────────────────────────────────┘    │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────────┐    │
│  │                 SHARED INFRASTRUCTURE                        │    │
│  │                                                              │    │
│  │  Memory:  Hot (SQLite) → Warm (JSON) → Cold (ledger)         │    │
│  │  Capital: Kelly engine → Portfolio → Circuit breaker         │    │
│  │  Bus:     message_bus (SQLite, priority queues)              │    │
│  │  Data:    web_scraper, api_poller, social_signal             │    │
│  │  KG:      kg_entities / kg_relationships                     │    │
│  └──────────────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────────────┘
```

Quick Start

```
# 1. Clone and enter the project
cd Solar-Concept-main

# 2. Run the setup script (installs deps, creates .env, initialises DB)
bash setup.sh

# 3. Edit .env with your API keys (minimum: OPENAI_API_KEY)
nano .env

# 4. Start all services
python main.py

# 5. (Optional) Start the swarm board UI
cd swarm-board
npm install
npm run dev
# Opens at http://localhost:5173
```

Port Map

PORT	SERVICE	FILE
5000	Human Gate API (approve/reject experiments)	`api/human_gate.py`
5001	GHL Webhook Server (inbound lead events)	`webhooks/ghl_handler.py`
5002	Voice AI Webhook (Retell + ElevenLabs)	`voice/call_handler.py`
5003	Dashboard API (swarm-board live feed)	`api/dashboard_api.py`
5173	Swarm Board React App (Vite dev server)	`swarm-board/`

Environment Variables Reference

VARIABLE	REQUIRED	DEFAULT	DESCRIPTION
`OPENAI_API_KEY`	Yes	—	GPT-4o key. System runs in mock mode without it.
`GHL_API_KEY`	Yes (for CRM)	—	GoHighLevel private API key
`GHL_LOCATION_ID`	Yes (for CRM)	—	GHL sub-account location ID
`GHL_PIPELINE_ID`	No	—	GHL pipeline ID for stage sync
`GHL_WEBHOOK_SECRET`	No	—	Validates inbound GHL webhook signatures
`SLACK_WEBHOOK_URL`	No	—	Incoming webhook URL for Slack alerts
`SLACK_BOT_TOKEN`	No	—	xoxb- token for Slack Web API
`SLACK_SIGNING_SECRET`	No	—	Verifies interactive Slack button payloads
`SLACK_DEFAULT_CHANNEL`	No	`#swarm-alerts`	Default Slack channel
`WEEKLY_BUDGET_AUD`	No	`500`	Total weekly experiment budget in AUD
`GATE_API_KEY`	No	—	Bearer token to protect human gate endpoints
`RETELL_API_KEY`	No	—	Retell AI API key for voice calls
`RETELL_DEFAULT_VOICE_ID`	No	`11labs-Adrian`	Default Retell/ElevenLabs voice
`ELEVENLABS_API_KEY`	No	—	ElevenLabs API key
`ELEVENLABS_DEFAULT_VOICE`	No	—	ElevenLabs voice ID
`PORT_HUMAN_GATE`	No	`5000`	Port for human gate Flask app
`PORT_GHL_WEBHOOKS`	No	`5001`	Port for GHL webhook Flask app
`PORT_VOICE_WEBHOOK`	No	`5002`	Port for voice AI Flask app
`PORT_DASHBOARD_API`	No	`5003`	Port for dashboard API Flask app

VARIABLE	REQUIRED	DEFAULT	DESCRIPTION
`VOICE_WEBHOOK_BASE_URL`	No	`http://localhost:5002`	Public URL Retell can reach
`TRANSFER_PHONE`	No	—	Phone number for human transfer calls
`DEFAULT_CLIENT_ID`	No	`default`	Fallback client ID
`DATABASE_PATH`	No	`swarm.db`	SQLite database file path
`LOG_LEVEL`	No	`INFO`	Python logging level
`HUBSPOT_API_KEY`	No	—	HubSpot private app token
`SALESFORCE_USERNAME`	No	—	Salesforce username
`SALESFORCE_PASSWORD`	No	—	Salesforce password
`SALESFORCE_SECURITY_TOKEN`	No	—	Salesforce security token
`SALESFORCE_CLIENT_ID`	No	—	Salesforce connected app client ID
`SALESFORCE_CLIENT_SECRET`	No	—	Salesforce connected app client secret
`IMAP_HOST`	No	—	IMAP server for email polling
`IMAP_USER`	No	—	IMAP username
`IMAP_PASS`	No	—	IMAP password
`IMAP_FOLDER`	No	`INBOX`	IMAP folder to poll
`FRONTEND_URL`	No	—	Production frontend URL for CORS allowlist

Documentation Index

FILE	WHAT IT COVERS
architecture.md	3-tier hierarchy, capital flow, memory system, message bus, scheduler jobs
agents.md	Every agent: purpose, functions, triggers, DB reads/writes
crm-integrations.md	GHL, HubSpot, Salesforce setup; crm_router API
capital-allocation.md	Kelly criterion, circuit breakers, 72-hour explore protocol
voice-ai.md	Retell + ElevenLabs integration, call flow, post-call processing
data-collection.md	Web scraper, API poller, social signal, pipeline processor
memory-database.md	Full SQLite schema, hot/warm/cold memory, message bus lifecycle
api-reference.md	Every Flask endpoint with method, params, response, curl examples
swarm-board.md	React app: how to run, Board/Overview tabs, board-state.json
setup-guide.md	Step-by-step setup from zero to running
client-onboarding.md	Non-technical guide for solar business owners
cost-tracking.md	API usage table, token tracking, cost projections
troubleshooting.md	Common errors and fixes

System Architecture

3-Tier Agent Hierarchy

```
TIER 1 — THE GENERAL
  master_agent.py
  Runs every 6 hours. Owns strategy.
  ├── Generates experiment ideas via GPT-4o (3 per cycle)
  ├── Scores each idea: market_signal + competitive_gap + execution_speed + revenue_path
  ├── Runs red team analysis (devil_score 1–10)
  ├── Routes: auto_proceed (>8.5) / human_gate (5.0–8.5) / auto_kill (<5.0)
  └── Allocates budget via Kelly Criterion → assigns bucket (exploit/explore/moonshot)

TIER 2 — DEPARTMENT HEADS (run daily at 09:00 UTC)
  research_agent.py   — manages research task queue, spawns prospect/market/competitor tasks
  content_agent.py    — manages content task queue (ad copy, email sequences, SMS, LinkedIn)
  analytics_agent.py  — analyses leads, experiments, budget burn; checks circuit breakers

TIER 3 — WORKERS (stateless, self-terminating)
  worker.py           — picks up next queued task, executes it, posts pheromone signal, exits
  qualification_agent — scores solar leads 1–10 via GPT-4o or rule-based fallback
  proposal_agent      — generates 3-section proposals, saves to proposals/
  solar_research_agent — profiles solar companies for outreach
  report_agent        — weekly client performance reports, saves to reports/
  red_team_agent      — devil's advocate analysis of experiment ideas

SUPPORT AGENTS
  scout_agent.py      — daily 08:00 UTC, discovers new solar company prospects
  mutation_engine.py  — Monday 22:30 UTC, evolves underperforming experiments via GPT-4o
  ab_tester.py        — daily 10:00 UTC, evaluates running A/B tests for winners
```

Capital Allocation Flow

```
confidence_score (0–10)
        |
        ▼
  score_experiment()        ← market_signal, competitive_gap, execution_speed, revenue_path
        |
        ▼
  red_team_analyse()        ← devil_score (1–10)
  adjust_confidence()       ← if devil_score > 6: confidence -= (devil_score - 6) × 0.5
        |
        ▼
  routing decision:
    > 8.5  → auto_proceed
    5.0–8.5 → human_gate  (Slack alert sent, approval required)
    < 5.0  → auto_kill
        |
        ▼
  calculate_budget()        ← 25% fractional Kelly
    p = 0.2 + (confidence/10 × 0.65)    # win probability
    f* = (b×p - q) / b                   # full Kelly (b=3.0 default)
    f_actual = f* × 0.25                 # 25% fractional
    budget = f_actual × WEEKLY_BUDGET_AUD
        |
        ▼
  assign_bucket()
    devil ≥ 7 AND confidence ≥ 7 → moonshot
    confidence ≥ 8 AND devil < 5 → exploit
    otherwise                    → explore
        |
        ▼
  can_allocate()            ← checks bucket remaining vs requested amount
        |
        ▼
  INSERT experiments        ← status: approved / pending
        |
        ▼
  circuit_breaker check     ← after each General cycle
```

Budget Buckets (of WEEKLY_BUDGET_AUD):

BUCKET	ALLOCATION	PURPOSE
exploit	60%	Scaling proven winners
explore	30%	72-hour test experiments
moonshot	10%	High-risk, high-reward ideas

Memory System

Hot Memory — SQLite (`swarm.db`)

Sub-second access to live state. All agents read/write directly.

- `get_active_experiments()` — experiments with status `approved` or `running`
- `get_pending_experiments()` — awaiting human approval
- `get_budget_used_this_week()` — SUM of budget_allocated, last 7 days
- `get_consecutive_failures()` — current failure streak from experiments table
- `enqueue_task()` / `get_next_task()` — task queue operations
- `post_pheromone()` / `get_active_pheromones()` — signal tracking
- `apply_pheromone_decay()` — 50% weight loss per day after 7 days
- `get_circuit_breaker_state()` — most recent unresolved circuit_breaker_log row

Warm Memory — JSON files (`memory/knowledge/`)

Richer structured records for pattern recognition. Slower than SQLite.

FILE	CONTENT
`experiments.json`	All completed experiment outcomes (status, revenue, ROI, learnings)
`learnings.json`	Actionable insights from past experiments, sorted by confidence
`verticals.json`	Market intelligence per vertical (solar_australia, etc.)

Key functions: `save_experiment_outcome()`, `save_learning()`, `get_winning_patterns()`, `get_all_learnings()`, `get_vertical_knowledge()`.

Cold Ledger — Append-only SQLite (`cold_ledger` table)

Every significant decision is written here and **never updated or deleted**. It is the system's audit trail.

Event types written to the ledger:

EVENT TYPE	TRIGGER
`EXPERIMENT_CREATED`	General creates new experiment
`EXPERIMENT_APPROVED`	Human or auto approval
`EXPERIMENT_KILLED`	Auto-kill or rejection
`CIRCUIT_BREAKER_YELLOW/ORANGE/RED`	Threshold breach
`LEAD_QUALIFIED`	Qualification agent scores a lead
`PHEROMONE_SIGNAL`	Worker posts a signal
`RED_TEAM_ANALYSIS`	Red team analysis completed
`RESEARCH_CYCLE`	Research department head run
`ANALYTICS_CYCLE`	Analytics department head run
`VOICE_CALL_COMPLETE`	Post-call processor finishes
`MUTATION_CYCLE`	Mutation engine completes

Message Bus Design

File: `bus/message_bus.py` — backed by the `message_bus` SQLite table.

All inter-agent communication goes through this bus. No direct agent-to-agent calls.

Priority order: `CRITICAL` → `HIGH` → `NORMAL` → `LOW`

Message types:

TYPE	PURPOSE
`TASK`	Request another agent to do work
`REPORT`	Return results of completed work
`ALERT`	Urgent notification requiring attention
`ACK`	Acknowledge receipt of a `requires_ack` message
`KILL`	Instruct agent to stop current work
`QUERY`	Request information
`RESPONSE`	Response to a QUERY

Message lifecycle: `queued` → `processing` → `complete` / `failed` / `expired`

Messages expire after 6 hours if unread (`expire_old_messages()` runs every 6 hours via scheduler).

Key functions:

```
bus.post(from_agent, to_queue, msg_type, payload, priority, ttl_cycles)
bus.receive(queue_name, msg_type=None)       # fetch + mark processing
bus.receive_all(queue_name, limit=20)        # drain queue
bus.complete(msg_id)
bus.fail(msg_id, reason)
bus.ack(original_msg_id, from_agent)
bus.queue_depth(queue_name)                  # counts per status
```

Data Flow Diagram

```
External World
      │
      ├── GHL Webhook (new lead) ──────────────────────────┐
      │                                                     │
      ├── Retell AI (inbound call) ────────────────────────┐  │
      │                                                    │  │
      └── Web Scraper / API Poller / Social Signal ──────┐    │  │
                                                         │    │  │
                                                         ▼    ▼  ▼
                                              ┌──────────────────┐
                                              │   SQLite (swarm.db) │
                                              │                  │
                                              │  collected_data  │
                                              │  leads           │
                                              │  call_logs       │
                                              │  message_bus     │
                                              │  experiments     │
                                              │  pheromone_signals │
                                              │  kg_entities     │
                                              │  crm_cache       │
                                              └──────────┬──────┘
                                                         │
                                                         │
                                                         │
                                                         │
                                                         ▼
                                                         ▼
                  ┌──────────────┐                ┌──────────────┐
                  │  APScheduler │                │  Flask APIs  │
                  │  (master_agent, │             │  :5000 human_gate │
                  │   scout_agent, │              │  :5001 ghl_handler │
                  │   analytics, etc)│            │  :5002 voice_ai │
                  └──────────────┘                │  :5003 dashboard │
                          │                       └──────────┬──────┘
                          │                                  │
                          ▼                                  ▼
                  ┌──────────────┐                ┌──────────────┐
                  │  OpenAI GPT-4o │              │  Swarm Board │
                  │  (generation, │               │  React :5173 │
                  │   scoring, etc) │             └──────────────┘
                  └──────────────┘
```

Scheduler Job Table

All times are UTC. Configured in `main.py` `setup_scheduler()`.

JOB ID	TRIGGER	WHAT IT DOES
`general`	Every 6h	The General's strategic planning cycle: generate ideas, score, route, allocate
`department_heads`	Daily 09:00	Run research_agent, content_agent, analytics_agent in sequence
`retrospective`	Monday 22:00	Weekly retrospective: analyse last 7 days, save learnings to warm memory
`pheromone_decay`	Daily 00:00	Apply 50% decay to pheromone signals older than 7 days
`scout_agent`	Daily 08:00	Discover new solar company prospects from scraped data and social signals
`research_engine`	Daily 06:00	Research orchestrator processes queued research tasks
`data_collection`	Every 4h	Web scraper, API poller, social signal, price monitor collect fresh data
`pipeline_processor`	Every 4h (+30min offset)	Deduplicate, enrich, and route signals from collected_data to message bus
`ab_evaluator`	Daily 10:00	Check running A/B tests; declare winners where sample size is sufficient
`mutation_engine`	Monday 22:30	Evolve underperforming experiments via GPT-4o; kill fatally bad ones
`explore_monitor`	Every 2h	Drive 72-hour explore protocol lifecycle; auto-kill expired experiments
`bus_expiry`	Every 6h	Expire stale messages on the message bus (>6h old queued messages)
`crm_sync`	Every 30min	Pull live CRM data into `crm_cache` table; update board-state.json

Agent Reference

Each agent is a Python module in `agents/`. All agents follow a common pattern: read from SQLite, call GPT-4o (or use a rule-based fallback), write results back to SQLite, and log to the cold ledger.

---

master_agent.py — The General (Tier 1)

Purpose: Strategic command. Generates experiment ideas, scores them, red-teams them, routes them, and allocates budget.

Scheduler trigger: Every 6 hours via `IntervalTrigger(hours=6)`

Key functions:

FUNCTION	PARAMS	RETURNS
`run()`	—	`list` of experiment dicts created this cycle
`_generate_ideas()`	—	`list` of 3 raw idea dicts from GPT-4o
`_process_idea(idea_data)`	`idea_data: dict`	Scored/routed experiment dict or `None`
`_mock_ideas()`	—	3 hardcoded mock ideas (used when no OpenAI key)

What it reads: `experiments` (active count, budget), `pheromone_signals` (via warm memory), `cold_ledger` (past learnings)

What it writes: `experiments` (new rows with status `approved` or `pending`)

Example output:

```
{
  "experiment_id": 42,
  "idea_text": "Cold email sequence to 50 solar SMEs in WA...",
  "confidence_score": 7.3,
  "devil_score": 4,
  "status": "approved",
  "bucket": "explore",
  "budget_aud": 37.50
}
```

Halts when: `is_halted()` returns True (circuit breaker Red state).

---

research_agent.py — Research Head (Tier 2)

Purpose: Manages the research task queue. Routes `research_*` job types to the appropriate handler, posts pheromone signals on completion, and queues routine market analysis if the queue is empty.

Scheduler trigger: Daily 09:00 UTC (as part of `_run_department_heads`)

Key functions:

FUNCTION	PARAMS	RETURNS
`run(max_tasks=5)`	`max_tasks: int`	`{tasks_processed, tasks_failed, signals_posted}`
`queue_prospect(company_name, suburb)`	`str, str`	`task_id: int`
`_dispatch_task(job_type, context)`	`str, dict`	Result dict
`_prospect_company(context)`	`{company_name, suburb}`	Research dict + pheromone data
`_analyse_market(context)`	`{vertical, keywords}`	Market analysis dict
`_analyse_competitor(context)`	`{competitor_name}`	Competitive intel dict

Job types handled: `research_prospect`, `research_market`, `research_competitor`

What it reads: `task_queue` (tier=3, job_type LIKE 'research_%')

What it writes: `task_queue` (status updates), `pheromone_signals` (via `post_pheromone`), `cold_ledger`

---

content_agent.py — Content Head (Tier 2)

Purpose: Manages the content task queue. Generates ad copy, email sequences, LinkedIn posts, and SMS templates via GPT-4o with Australian B2B copywriting style.

Scheduler trigger: Daily 09:00 UTC (as part of `_run_department_heads`)

Key functions:

FUNCTION	PARAMS	RETURNS
`run(max_tasks=5)`	`max_tasks: int`	`{tasks_processed, tasks_failed}`
`_generate_ad_copy(context)`	`{target_audience, pain_point, offer, url}`	`{ad_copy, status}`
`_generate_email_sequence(context)`	`{prospect_name, company, pain_point}`	`{email_1, email_2, email_3, status}`
`_generate_linkedin_post(context)`	`{topic}`	`{post, status}`
`_generate_sms(context)`	`{name}`	`{sms, status}`

Job types handled: `content_ad_copy`, `content_email`, `content_linkedin`, `content_sms`

System prompt: Australian B2B copywriter, plain and punchy, no corporate speak, Australian English spelling.

analytics_agent.py — Analytics Head (Tier 2)

Purpose: Daily analytics cycle. Analyses leads and experiments, tracks budget burn rate, and checks circuit breakers.

Scheduler trigger: Daily 09:00 UTC (as part of `_run_department_heads`)

Key functions:

FUNCTION	PARAMS	RETURNS
`run()`	—	Full analytics summary dict
`_analyse_leads()`	—	`{total_7d, qualified, hot_leads, converted, conversion_rate, avg_score}`
`_analyse_experiments()`	—	`{total_7d, complete, killed, running, success_rate, total_revenue_aud, roi}`
`_analyse_budget()`	—	`{weekly_budget_aud, used_aud, remaining_aud, burn_rate, on_track}`
`_check_circuit_breakers(budget_stats)`	`dict`	Circuit breaker result
`get_conversion_stats()`	—	Full lead pipeline breakdown

What it reads: `leads` (last 7 days), `experiments` (last 7 days), `circuit_breaker_log`

What it writes: `cold_ledger` (ANALYTICS_CYCLE event), may trigger `circuit_breaker_log` entries

---

worker.py — Tier 3 Worker

Purpose: Stateless task executor. Picks up one task from the queue, executes it, posts a pheromone signal, and terminates. All state is in the database.

Trigger: Called directly by department heads, or via `python worker.py [task_id]`

Key functions:

FUNCTION	PARAMS	RETURNS
`run_task(task_id=None)`	`task_id: int \`	None`
`_execute(job_type, context)`	`str, dict`	Result dict with `signal_type`, `topic`, `strength`

Job types handled:

JOB TYPE	HANDLER
`research_prospect`	`solar_research_agent.research()`
`research_market`	Placeholder market analysis
`content_ad_copy`	`content_agent._generate_ad_copy()`
`content_email`	`content_agent._generate_email_sequence()`
`content_linkedin`	`content_agent._generate_linkedin_post()`
`content_sms`	`content_agent._generate_sms()`
`solar_qualify_lead`	`qualification_agent.qualify()`
`solar_research_company`	`solar_research_agent.research()`
`solar_generate_proposal`	`proposal_agent.generate()`
`generic_test`	Returns success dict

Pheromone signal strength by outcome:

- Successful prospect (score ≥ 6) → `POSITIVE`, strength = score/10
- Failed task → `NEGATIVE`, strength = 0.3
- Neutral tasks → `NEUTRAL`, strength = 0.5

---

scout_agent.py — Prospect Hunter

Purpose: Daily autonomous discovery of Australian solar companies likely to need CRM automation. Fuses three data sources, scores candidates, queues top 10 for deep research, and saves top 5 as opportunities.

Scheduler trigger: Daily 08:00 UTC

Key functions:

FUNCTION	PARAMS	RETURNS
`run()`	—	`{prospects_found, queued_for_research, opportunities_saved}`
`_gather_candidates()`	—	Raw candidates from all sources
`_score_candidate(candidate)`	`dict`	Candidate with `scout_score` 0–10
`_queue_for_research(prospects)`	`list`	Count queued to message bus
`_save_opportunities(prospects)`	`list`	Count saved to opportunities table

Scoring logic:

SOURCE	BASE SCORE	HOT SIGNAL BONUS
`installer_registry`	3.0	+1.5 per hot keyword hit
`social_signal`	5.0	+2.0 if "hiring" present
`knowledge_graph`	4.0	+1.5 per hot keyword hit

Hot signals: hiring, scaling, crm, manual process, follow up, new office

If the company already appears in the `leads` table, `-3.0` is applied to avoid double-handling.

What it reads: `collected_data` (last 2 days), `kg_entities`, `leads`

What it writes: `message_bus` (TASK to research_queue), `opportunities` table

---

mutation_engine.py — Strategy Evolution

Purpose: Monday retrospective job. Finds underperforming experiments (score < 5 or 3+ consecutive failures), generates 2 mutated variants per loser using GPT-4o, and either submits them as new experiments or kills the parent.

Scheduler trigger: Monday 22:30 UTC

Key functions:

FUNCTION	PARAMS	RETURNS
`run()`	—	`{analysed, mutations_created, killed}`
`_find_underperformers()`	—	Up to 5 experiments with score < 5 or consecutive_failures ≥ 3
`_mutate(experiment)`	`dict`	`{mutations, kill_recommendation, kill_reason}`
`_submit_mutations(mutations, parent)`	`list, dict`	Inserts new experiment rows with status `queued`
`_kill_experiment(experiment)`	`dict`	Updates experiment status to `killed`

Mutation output format from GPT-4o:

```
{
  "mutations": [{
    "name": "Experiment v2 — Faster Follow-up",
    "hypothesis": "Calling within 5 minutes increases conversion",
    "changes_from_parent": ["Reduce SMS-to-call delay from 30min to 5min"],
    "expected_improvement": "Industry data shows 5min = 9x higher connect rate",
    "confidence": 0.72,
    "estimated_revenue": 3000,
    "risk_level": "low"
  }],
  "kill_recommendation": false,
  "kill_reason": ""
}
```

---

ab_tester.py — A/B Test Manager

Purpose: Creates, tracks, and evaluates A/B tests. Declares winners when statistical thresholds are met and emits pheromone signals to reinforce winning strategies.

Scheduler trigger: Daily 10:00 UTC (`evaluate_tests()`)

Key functions:

FUNCTION	PARAMS	RETURNS
`create_test(name, hypothesis, variant_a, variant_b, metric, experiment_id)`	Various	`test_id: str`
`record_event(test_id, variant, event_type, value=1.0)`	Various	None
`evaluate_tests()`	—	`{evaluated, winners_found, tests_extended}`
`get_summary()`	—	Status counts dict

Evaluation rules:

- Minimum 30 impressions per variant before evaluation
- Less than 10% lift → `no_winner`
- ≥ 10% lift → winner is variant with higher conversion rate
- Winner emits `ab_winner` pheromone signal at strength 0.8

Metric options: `conversion_rate`, `reply_rate`, `booking_rate`

What it reads/writes: `ab_tests` table

---

qualification_agent.py — Lead Scorer

Purpose: Scores solar leads 1–10 using GPT-4o. Triggers outbound Retell calls for hot leads (score ≥ 7 with phone number). Sends Slack alerts.

Trigger: Called by `ghl_handler.py` on each new webhook lead, or as worker task `solar_qualify_lead`

Key functions:

FUNCTION	PARAMS	RETURNS	
`qualify(lead_data, lead_id=None)`	`dict, int |	None`	`{score, reason, recommended_action, key_signals, risk_flags}`
`_ai_score(lead_data)`	`dict`	GPT-4o scoring result	
`_rule_based_score(lead_data)`	`dict`	Rule-based fallback scoring	
`_trigger_outbound_call(lead_data, lead_id)`	`dict, int`	Fires Retell call if configured	

Scoring criteria (rule-based fallback):

SIGNAL	POINTS
Homeowner (owner)	+3
Bill > $300/mo	+3
Bill $200–300/mo	+2
Bill $150–200/mo	+1
Ideal roof (tile/colorbond < 15yr)	+2
High sun state (QLD/WA/SA/NSW/NT)	+2

Routing: score ≥ 7 → `call_now`, 5–6 → `nurture`, ≤ 4 → `disqualify`

What it writes: `leads` (qualification_score, score_reason, recommended_action, status), `cold_ledger`, `call_logs` (if outbound call initiated)

proposal_agent.py — Proposal Generator

Purpose: Generates 3-section business proposals for solar company prospects and saves them to `proposals/`.

Trigger: Worker task `solar_generate_proposal`, or called directly via CLI

Key functions:

FUNCTION	PARAMS	RETURNS
`generate(client_name, pain_points, current_process, estimated_leads_per_month)`	Various	`{file_path, client_name, proposal_text, generated_at}`

Proposal structure:

1. Current State — problem quantified with estimated cost of lost leads (25% × avg $8,000–12,000 per system)
2. Future State — what the AI automation system delivers, with specific metrics
3. Investment — three tiers: Starter $1,500/mo, Growth $2,000/mo (recommended), Premium $2,500/mo

Output: Saved as `proposals/{ClientName}_{YYYYMMDD}.txt`

---

solar_research_agent.py — Company Profiler

Purpose: Given a solar company name and suburb, generates a structured research profile for personalised outreach.

Trigger: Research task `research_prospect`, or called directly

Key functions:

FUNCTION	PARAMS	RETURNS
`research(company_name, suburb)`	`str, str`	Full research profile dict
`batch_research(companies)`	`list[{company_name, suburb}]`	List sorted by score DESC
`get_outreach_message(research_result, sender_name)`	`dict, str`	Personalised email/LinkedIn message

Research profile fields: company_name, suburb, website, google_maps_present, estimated_staff, owner_name, review_summary, admin_pain_points, crm_signals (has_booking_link, has_contact_form, has_chat_widget), outreach_angle, score (1–10)

---

report_agent.py — Client Report Generator

Purpose: Generates weekly performance reports for solar clients and saves them to `reports/`.

Trigger: Called directly via CLI or scheduled via `generate_all_clients()`

Key functions:

FUNCTION	PARAMS	RETURNS
`generate(client_name, date_range=7)`	`str, int`	`{file_path, report_text, stats, generated_at}`
`generate_all_clients(days=7)`	`int`	List of report results

Stats collected: total_leads, qualified (≥5/10), hot_leads (≥7/10), contacted, converted, disqualified, avg_score, contact_rate_pct, conversion_rate_pct, leads_not_lost (estimated 25%), estimated_revenue_protected (leads_not_lost × $8,000)

Output: Saved as `reports/{ClientName}_report_{YYYYMMDD}.txt`

---

red_team_agent.py — Devil's Advocate

Purpose: Analyses experiment ideas for failure modes. Returns a `devil_score` (1–10, higher = more flaws) that triggers automatic confidence downgrades in the General.

Trigger: Called by `master_agent._process_idea()` for every experiment idea

Key functions:

FUNCTION	PARAMS	RETURNS	
`analyse(idea_text, experiment_id=None)`	`str, int \	None`	`{devil_score, failure_modes, summary}`
`adjust_confidence(confidence_score, devil_score)`	`float, int`	Adjusted confidence score	

Confidence adjustment formula:

- If `devil_score > 6`: `penalty = (devil_score − 6) × 0.5`, `adjusted = confidence − penalty`
- Example: confidence 7.5, devil_score 8 → penalty 1.0 → adjusted 6.5

Failure modes returned (3 per analysis): ranked by severity, each with mode title and 2-sentence explanation

What it writes: `cold_ledger` (RED_TEAM_ANALYSIS event)

Memory & Database Reference

Overview

All persistent state is stored in a single SQLite file (`swarm.db` by default, configurable via `DATABASE_PATH`). The database uses WAL (Write-Ahead Logging) journal mode for concurrent access safety.

Three memory tiers serve different access patterns:

TIER	STORAGE	ACCESS SPEED	PURPOSE
Hot	SQLite (`swarm.db`)	Sub-second	Live state: experiments, tasks, leads, signals
Warm	JSON files (`memory/knowledge/`)	~10ms	Pattern recognition, retrospective analysis
Cold	SQLite (`cold_ledger` table)	Write-once	Immutable audit trail; never updated

Full SQLite Schema

Schema defined in `memory/database.py` `init_db()`. All tables created with `IF NOT EXISTS`.

`experiments`

Core experiment tracking table.

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
id	INTEGER	PK AUTOINCREMENT	
created_at	TEXT	DEFAULT datetime('now')	
idea_text	TEXT	NOT NULL	Full experiment description
vertical	TEXT		e.g. solar_australia
bucket	TEXT	CHECK: exploit/explore/moonshot	Portfolio bucket
confidence_score	REAL		0–10 composite score
devil_score	REAL		Red team score 1–10
kelly_fraction	REAL		Calculated Kelly fraction
budget_allocated	REAL	DEFAULT 0	AUD allocated
status	TEXT	DEFAULT 'pending', CHECK enum	pending/approved/running/complete/killed/rejected
revenue_generated	REAL	DEFAULT 0	Actual revenue from this experiment
roi	REAL		Return on investment
learnings	TEXT		Free-text learnings
failure_mode	TEXT		Why it failed
approved_by	TEXT		'auto' or username
approved_at	TEXT		ISO timestamp
completed_at	TEXT		ISO timestamp
paid_spend_activated	INTEGER	DEFAULT 0	72hr explore protocol flag
explore_phase	TEXT		Current phase name

`task_queue`

Work queue for Tier 3 workers.

COLUMN	TYPE	DESCRIPTION
id	INTEGER PK	
created_at	TEXT	
job_type	TEXT NOT NULL	e.g. research_prospect, content_ad_copy
priority	INTEGER	1=highest, 10=lowest. Tier 2 uses 3–7
context_payload	TEXT	JSON dict of task parameters
assigned_to	TEXT	Worker identifier
status	TEXT	queued/running/complete/failed
output	TEXT	JSON result from worker
completed_at	TEXT	
tier	INTEGER	DEFAULT 3

`pheromone_signals`

Reinforcement signals from workers that bias future experiment generation.

COLUMN	TYPE	DESCRIPTION
id	INTEGER PK	
created_at	TEXT	
signal_type	TEXT	POSITIVE/NEGATIVE/NEUTRAL
topic	TEXT	e.g. solar_prospecting, lead_quality
vertical	TEXT	e.g. solar_australia
strength	REAL	0.0–1.0
channel	TEXT	Source channel
experiment_id	INTEGER	Associated experiment
decay_factor	REAL	DEFAULT 1.0; reduced by apply_pheromone_decay()

`cold_ledger`

Append-only audit trail. Records are never updated or deleted.

COLUMN	TYPE	DESCRIPTION
id	INTEGER PK	
created_at	TEXT	
event_type	TEXT NOT NULL	e.g. EXPERIMENT_CREATED, LEAD_QUALIFIED
event_data	TEXT	JSON dict of event details
experiment_id	INTEGER	Associated experiment if applicable
agent_id	TEXT	Which agent triggered this
human_involved	INTEGER	0/1

`leads`

Solar lead records.

COLUMN	TYPE	DESCRIPTION
id	INTEGER PK	
created_at	TEXT	
source	TEXT	ghl_webhook/manual/form
name	TEXT	
phone	TEXT	
email	TEXT	
suburb	TEXT	
state	TEXT	AU state code
homeowner_status	TEXT	owner/renter/unknown
monthly_bill	REAL	AUD per month
roof_type	TEXT	tile/colorbond/flat/metal
roof_age	INTEGER	Years
qualification_score	REAL	1–10 AI score
score_reason	TEXT	2-sentence explanation
recommended_action	TEXT	call_now/nurture/disqualify
pipeline_stage	TEXT	GHL stage name
status	TEXT	DEFAULT 'new'
contacted_at	TEXT	When first contacted
converted_at	TEXT	When converted to client
client_account	TEXT	Solar company client identifier
notes	TEXT	Appended call notes, outcomes

`circuit_breaker_log`

Circuit breaker events.

COLUMN	TYPE	DESCRIPTION
id	INTEGER PK	
triggered_at	TEXT	
level	TEXT	yellow/orange/red
reason	TEXT	Human-readable reason
consecutive_failures	INTEGER	Failure streak at trigger time
budget_burn_rate	REAL	Burn rate at trigger time
resolved_at	TEXT	NULL if still active
resolved_by	TEXT	Who reset it

`message_bus`

Inter-agent communication bus.

COLUMN	TYPE	DESCRIPTION
id	INTEGER PK	
created_at	TEXT	
msg_id	TEXT UNIQUE	e.g. msg_abc123def456
from_agent	TEXT NOT NULL	Sender identifier
to_queue	TEXT NOT NULL	Destination queue name
msg_type	TEXT NOT NULL	TASK/REPORT/ALERT/ACK/KILL/QUERY/RESPONSE
priority	TEXT	CRITICAL/HIGH/NORMAL/LOW
payload	TEXT	JSON dict
reply_to	TEXT	msg_id this responds to
ttl_cycles	INTEGER	DEFAULT 3; expire after N cycles
requires_ack	INTEGER	0/1
status	TEXT	queued/processing/complete/failed/expired
acked_at	TEXT	
completed_at	TEXT	

`research_findings`

Results from the research engine.

COLUMN	TYPE	DESCRIPTION
research_id	TEXT UNIQUE	Unique research request ID
research_type	TEXT	prospect/market/competitive/technical
query	TEXT	The research query
requested_by	TEXT	Agent that requested it
status	TEXT	pending/in_progress/complete/failed
findings	TEXT	JSON research results
confidence	REAL	0.0–1.0
sources_count	INTEGER	Number of sources consulted
opportunities_found	INTEGER	Count of opportunities identified
expires_at	TEXT	Cached findings expiry
completed_at	TEXT	

`kg_entities`

Knowledge graph entities.

COLUMN	TYPE	DESCRIPTION
entity_id	TEXT UNIQUE	e.g. ent_abc123def
entity_type	TEXT	company/person/tool/market/location
name	TEXT	Entity name
properties	TEXT	JSON dict of attributes
confidence	REAL	0.0–1.0
source	TEXT	Where discovered
mention_count	INTEGER	How many times seen
first_seen	TEXT	
last_seen	TEXT	

`kg_relationships`

Knowledge graph edges between entities.

COLUMN	TYPE	DESCRIPTION
rel_id	TEXT UNIQUE	e.g. rel_xyz789
from_entity	TEXT	Source entity_id
to_entity	TEXT	Target entity_id
rel_type	TEXT	USES/COMPETES_WITH/LOCATED_IN/EMPLOYS/PARTNERS_WITH/REFERS
properties	TEXT	JSON dict
confidence	REAL	

`collection_sources`

Registry of all data collection sources.

COLUMN	TYPE	DESCRIPTION
source_id	TEXT UNIQUE	e.g. cec_registry_wa
name	TEXT	Human-readable name
source_type	TEXT	web_scrape/api_poll/social/price_monitor
url_template	TEXT	URL pattern
config	TEXT	JSON configuration
collection_frequency	TEXT	daily/hourly/weekly
frequency_hours	INTEGER	DEFAULT 24
priority	TEXT	NORMAL/HIGH/LOW
active	INTEGER	1=enabled
last_collected	TEXT	
health_status	TEXT	healthy/degraded/dead
error_count	INTEGER	Consecutive error count

`collected_data`

Raw data from all collection sources.

COLUMN	TYPE	DESCRIPTION
source_id	TEXT	References collection_sources
source_type	TEXT	Inherited from source
record_type	TEXT NOT NULL	Data category
record_key	TEXT	Optional dedup key
raw_data	TEXT	Original payload JSON
normalized_data	TEXT	Processed payload JSON
quality_score	REAL	0.0–1.0
dedup_hash	TEXT	Fingerprint for deduplication
processed	INTEGER	0/1
normalized	INTEGER	0/1

`time_series`

Metric recording for trend detection.

COLUMN	TYPE	DESCRIPTION
recorded_at	TEXT	
series_name	TEXT	e.g. ctr, cpl, lead_volume
entity_id	TEXT	Associated experiment or source
value	REAL	Metric value
unit	TEXT	e.g. AUD, percent, count
tags	TEXT	JSON dict

`opportunities`

Opportunities discovered by scout/research agents.

COLUMN	TYPE	DESCRIPTION
opportunity_id	TEXT UNIQUE	e.g. opp_abc123
opp_type	TEXT	prospect/market_gap/partnership
title	TEXT	
description	TEXT	
estimated_monthly_revenue_aud	REAL	
effort_score	REAL	1–10
speed_score	REAL	1–10
risk_score	REAL	1–10
overall_score	REAL	
status	TEXT	new/researching/queued/active/passed/killed
source_agent	TEXT	
research_id	TEXT	
experiment_id	INTEGER	
evidence	TEXT	JSON dict

`ab_tests`

A/B test tracking.

COLUMN	TYPE	DESCRIPTION
test_id	TEXT UNIQUE	e.g. abt_abc123
name	TEXT	
hypothesis	TEXT NOT NULL	
variant_a	TEXT	JSON config for control
variant_b	TEXT	JSON config for treatment
metric	TEXT	conversion_rate/reply_rate/booking_rate
status	TEXT	running/complete/inconclusive
a_impressions / b_impressions	INTEGER	
a_conversions / b_conversions	INTEGER	
a_clicks / b_clicks	INTEGER	
winner	TEXT	a/b/no_winner
winner_stats	TEXT	JSON stats dict
completed_at	TEXT	

`call_logs`

Voice AI call records.

COLUMN	TYPE	DESCRIPTION
call_id	TEXT UNIQUE	Retell call ID
client_id	TEXT	Solar company client
from_phone	TEXT	Caller number
to_phone	TEXT	Called number
agent_id	TEXT	Retell agent ID
status	TEXT	started/active/complete/failed
duration_seconds	INTEGER	
recording_url	TEXT	
outcome	TEXT	booked_assessment/callback_requested/etc
lead_score	REAL	
summary	TEXT	GPT-4o call summary
transcript_turns	INTEGER	

`api_usage`

API cost tracking.

COLUMN	TYPE	DESCRIPTION
service	TEXT	openai/retell/elevenlabs
operation	TEXT	chat.completion/phone_call/tts
model	TEXT	gpt-4o/etc
units	REAL	Tokens, seconds, characters
unit_type	TEXT	tokens/seconds/characters
cost_usd	REAL	Estimated USD cost
call_id	TEXT	Associated voice call
client_id	TEXT	Associated solar client
metadata	TEXT	JSON

`crm_cache`

CRM data cached by `crm_sync` every 30 minutes.

COLUMN	TYPE	DESCRIPTION
cached_at	TEXT	
cache_key	TEXT UNIQUE	e.g. contact_abc123, pipeline_stage_xyz, metrics_summary
cache_value	TEXT	JSON serialised CRM data

Cache TTL: Data is refreshed every 30 minutes by the `crm_sync` scheduler job. The dashboard API reads from this cache rather than hitting the CRM directly on every request.

Other Tables

TABLE	PURPOSE
`users`	Multi-user auth (email, password_hash, role: owner/admin/client)
`auth_tokens`	JWT revocation list
`company_profiles`	One row per solar SME client (name, ABN, contact_email, retell_agent_id, etc.)
`api_keys`	Client embed / webhook auth keys
`app_settings`	Runtime config overrides (key, value, category)
`email_logs`	Email processing records (intent, score, draft_reply_queued)

How Migrations Work (`_apply_migrations`)

The `_apply_migrations()` function in `memory/database.py` runs on every startup, immediately after `init_db()`. It adds columns that exist in application code but may be missing from older database schemas.

Each migration is an `ALTER TABLE ADD COLUMN` statement. SQLite will raise an exception if the column already exists, which is caught and silently ignored.

```
migrations = [
    ("ab_tests", "name TEXT"),
    ("ab_tests", "experiment_id INTEGER"),
    ("kg_entities", "properties TEXT"),
    ("experiments", "paid_spend_activated INTEGER DEFAULT 0"),
    ("experiments", "explore_phase TEXT"),
    # ... etc
]
```

This design means the database schema is always at least as current as the code, with no manual migration steps required.

Hot Memory API Reference

File: `memory/hot_memory.py`

Experiments

```
get_active_experiments()                    # status IN ('approved','running')
get_pending_experiments()                   # status = 'pending', sorted by confidence DESC
get_experiment(experiment_id)               # single row dict
update_experiment_status(id, status, extra) # update status + optional extra fields
get_budget_used_this_week()                 # float AUD
get_consecutive_failures()                  # int — current failure streak
```

Task Queue

```
enqueue_task(job_type, context, priority=5, tier=3)  # returns task_id int
get_next_task(tier=None)                              # highest-priority queued task
complete_task(task_id, output)
fail_task(task_id, reason)
```

Pheromone Signals

```
post_pheromone(signal_type, topic, strength, vertical, channel, experiment_id)
get_active_pheromones(topic=None)    # decay_factor > 0.01
apply_pheromone_decay()              # 50% decay to signals older than 7 days
```

Circuit Breaker

```
get_circuit_breaker_state()   # most recent unresolved log row; {level, active, ...}
get_swarm_summary()           # {active_experiments, pending_approval, budget_used_aud, ...}
```

Warm Memory API Reference

File: `memory/warm_memory.py`

Files stored in `memory/knowledge/`:

```
save_experiment_outcome(experiment_id, idea_text, outcome)
save_learning(topic, insight, source, confidence=0.7)
get_learnings_for_topic(topic)     # sorted by confidence DESC
get_all_learnings()                # full list
get_winning_patterns()             # complete experiments with positive ROI
get_experiment_history(limit=50)   # sorted by recorded_at DESC
update_vertical_knowledge(vertical, data)
get_vertical_knowledge(vertical)
```

Cold Ledger API Reference

File: `memory/cold_ledger.py`

```
log_event(event_type, event_data, experiment_id, agent_id, human_involved)
log_experiment_created(experiment_id, idea_text, confidence, agent_id)
log_experiment_approved(experiment_id, approved_by, budget)
log_experiment_killed(experiment_id, reason, agent_id)
log_circuit_breaker(level, reason, data)
log_lead_qualified(lead_id, score, action)
log_pheromone_signal(signal_type, topic, strength, experiment_id)
get_recent_events(limit=50)
get_events_by_type(event_type, limit=100)
```

Knowledge Graph API Reference

File: `storage/knowledge_graph.py`

```
upsert_entity(name, entity_type, properties, source, confidence)  # create or update; returns entity_
upsert_relationship(from_id, to_id, rel_type, properties, confidence)  # returns rel_id
get_entity(name, entity_type=None)
get_neighbors(entity_id, rel_type=None)
search_entities(entity_type, limit=20)  # sorted by mention_count DESC
get_graph_summary()  # {entities: {type: count}, relationships: {type: count}}
```

Upsert pattern: If an entity with the same `name` and `entity_type` already exists, its `properties`, `confidence`, `last_seen`, and `mention_count` are updated. If not, a new entity with a generated `entity_id` is created.

Data Collection Engine

Overview

The data collection engine runs on a 4-hour cycle (scheduler job `data_collection`). It collects fresh market data from four sources, stores raw records in `collected_data`, then the pipeline processor runs 30 minutes later to normalise, deduplicate, and route high-value signals to the message bus.

Files:

- `data_collection/orchestrator.py` — coordinates collection across all active sources
- `data_collection/agents/web_scraper.py` — HTML scraping
- `data_collection/agents/api_poller.py` — GHL API polling
- `data_collection/agents/social_signal.py` — social media signal detection
- `data_collection/agents/price_monitor.py` — CPL benchmark tracking
- `data_collection/pipeline/processor.py` — deduplication, enrichment, bus routing

Collection Sources

Source Registry

All active sources are stored in the `collection_sources` table. Each source has a `source_type` that determines which agent handles it.

SOURCE_TYPE	AGENT	WHAT IT COLLECTS
`web_scrape`	`web_scraper.py`	Solar installer registries, directories
`api_poll`	`api_poller.py`	GHL contacts and pipeline opportunities
`social`	`social_signal.py`	LinkedIn buying signals
`price_monitor`	`price_monitor.py`	CPL benchmarks and lead price trends

1. Web Scraper (`web_scraper.py`)

Scrapes public HTML sources for solar company data. Requires `requests` and `beautifulsoup4`.

`collect(source)` → `{success, records, signals, error}`

The `source` dict must include `config.url` and `config.data_type`. Supported data types:

DATA_TYPE	PARSER	FIELDS EXTRACTED
`solar_installer`	`_parse_installer_table()`	company_name, licence_number, state, source_url
`generic`	Link extractor	url, text, source_url

Falls back to mock data (`_mock_records()`) when requests/bs4 are unavailable or the URL is blocked. Mock data returns 3 sample installer records for development.

2. API Poller (`api_poller.py`)

Polls GoHighLevel REST API for contacts and pipeline opportunities. Falls back to mock data when `GHL_API_KEY` is not configured.

`collect(source)` → `{success, records, signals, error}`

The `source` dict `config.endpoint` field determines what to fetch:

ENDPOINT	WHAT'S RETURNED
`contacts`	GHL contact records
`opportunities`	Pipeline opportunity records
Other	Raw response

The old GHL API base URL used here is `https://rest.gohighlevel.com/v1` (note: this differs from the `ghl_client.py` which uses `https://services.leadconnectorhq.com`).

3. Social Signal (`social_signal.py`)

Monitors LinkedIn and other social platforms for buying signals — hiring posts, scaling announcements, and pain-point complaints that indicate a company is ready for CRM automation.

`collect(source)` → `{success, records, signals, error}`

The `source` dict `config.platform` and `config.query` fields configure what to scan.

Signal keywords monitored:

```
SIGNAL_KEYWORDS = [
    "hiring", "scaling", "growth", "looking for", "need help",
    "manual process", "too many leads", "follow up", "crm",
    "solar install", "new office", "expanding", "franchise",
]
```

Classification logic:

1. Count keyword hits in post text
2. Base signal strength: 0 hits → low, 1–2 hits → medium, 3+ hits → high
3. If OpenAI is configured and score ≥ 2: GPT-4o refines the classification with context

Note: Live social API access requires paid credentials (LinkedIn API, etc.). The current implementation uses mock data with realistic examples for development.

4. Price Monitor (`data_collection/agents/price_monitor.py`)

Tracks CPL (cost per lead) benchmarks and lead price trends in the Australian solar market. Results are stored in the `time_series` table for trend detection.

Pipeline Processor (`processor.py`)

`process_batch(since_minutes=60)` → `{processed, deduplicated, signals_posted}`

The processor runs every 4 hours (30 minutes after the collection cycle) and processes all unprocessed records from `collected_data`.

Step 1: Fetch Unprocessed Records

```
SELECT * FROM collected_data
WHERE pipeline_processed IS NULL
AND collected_at >= datetime('now', '-{since_minutes} minutes')
ORDER BY collected_at ASC
LIMIT 200
```

Step 2: Deduplication

Each record gets a fingerprint from key fields:

```
fingerprint = "|".join([
    data.get("company_name", "").lower(),
    data.get("url", "").lower(),
    data.get("keyword", "").lower(),
    data.get("id", "").lower(),
])
```

Records with the same fingerprint in the current batch are skipped.

Step 3: Enrichment Scoring

SOURCE_TYPE	CONDITION	ENRICHED_SCORE
`social`	signal_strength == "high"	9
`social`	signal_strength == "medium"	6
`web_scrape`	company_name present	5
`api_poll`	any	7

Step 4: Signal Extraction

Records with `enriched_score ≥ 5` are converted to signals:

SOURCE_TYPE	SIGNAL_TYPE	FIELDS
`social`	`social_buying_signal`	company, strength, evidence, url
`web_scrape`	`new_prospect_found`	company, state, licence
`api_poll`	`ghl_contact_update`	contact_id, name

Step 5: Bus Routing

Signals with score ≥ 5 are posted to `research_queue` on the message bus:

```
message_bus.post(
    from_agent="pipeline_processor",
    to_queue="research_queue",
    msg_type="ALERT",
    payload={"signal": signal, "source_record_id": rec["record_id"]},
    priority="HIGH" if score >= 8 else "NORMAL",
)
```

Step 6: Mark Processed

Each processed record gets `pipeline_processed` stamped with the current timestamp.

How to Add a New Collection Source

1. **Create an agent file** in `data_collection/agents/your_source.py`

Implement a `collect(source: dict) -> dict` function that:

- Reads config from `source["config"]`
- Returns `{success: bool, records: int, signals: int}`
- Writes records to `collected_data` table

1. **Register the source type** in `data_collection/orchestrator.py`

Add your `source_type` to the dispatcher dict that maps types to agent modules.

1. **Insert a row** into `collection_sources` table:

`` ` ``python

from memory.database import insert, json_payload

insert("collection_sources", {

"source_id": "my_source_001",

"name": "My Data Source",

"source_type": "your_type", # must match your agent's handling

"url_template": "https://...",

"config": json_payload({"url": "https://...", "data_type": "my_type"}),

"collection_frequency": "daily",

"frequency_hours": 24,

"priority": "NORMAL",

"active": 1,

})

`` ` ``

`collected_data` Table Schema

COLUMN	TYPE	DESCRIPTION
`id`	INTEGER PK	Auto-increment
`collected_at`	TEXT	Timestamp (UTC)
`source_id`	TEXT NOT NULL	References collection_sources.source_id
`source_type`	TEXT	web_scrape / api_poll / social / price_monitor
`record_type`	TEXT NOT NULL	Data category (solar_installer, contact, etc.)
`record_key`	TEXT	Optional dedup key
`raw_data`	TEXT	Original JSON from source
`normalized_data`	TEXT	Processed/normalised JSON
`quality_score`	REAL	0.0–1.0 quality rating
`dedup_hash`	TEXT	Fingerprint for deduplication
`processed`	INTEGER	0/1 — has been through pipeline
`normalized`	INTEGER	0/1 — has been normalised

Note: The `pipeline_processor.py` also uses a `pipeline_processed` column (added via migration) to track which records have been through the pipeline processor specifically.

Processing Frequency

JOB	FREQUENCY	OFFSET
Data collection	Every 4 hours	Starts immediately
Pipeline processor	Every 4 hours	Starts 30 minutes after collection
Scout agent	Daily 08:00 UTC	Reads from collected_data (last 2 days)
Research engine	Daily 06:00 UTC	Processes research_queue messages from bus

The 30-minute offset between collection and processing ensures fresh records are available before processing begins.

API Endpoint Reference

The system exposes four Flask services on separate ports. All use JSON request/response bodies unless noted. Security headers (`X-Content-Type-Options`, `X-Frame-Options`, `Cache-Control: no-store`) are attached to every response.

Port Map

PORT	SERVICE	FILE
5000	Human Gate API	`api/human_gate.py`
5001	GHL Webhook Server	`webhooks/ghl_handler.py`
5002	Voice Webhook Server	`voice/call_handler.py`
5003	Dashboard API	`api/dashboard_api.py`

Human Gate API — Port 5000

Authentication: All endpoints except `/health` require:

```
Authorization: Bearer <GATE_API_KEY>
```

Set `GATE_API_KEY` in `.env`. If not set, auth is bypassed with a warning (acceptable for local dev; always set in production).

Rate limit: 20 requests per minute per IP on approve/reject endpoints.

GET /health

Public — no auth required.

Response:

```
{
  "status": "ok",
  "service": "human-gate",
  "circuit_breaker": "green",
  "halted": false,
  "timestamp": "2026-03-09T08:00:00"
}
```

```
curl http://localhost:5000/health
```

GET /pending

List all experiments awaiting human review (status = 'pending').

Response:

```
{
  "count": 2,
  "experiments": [
    {
      "id": 42,
      "idea_text": "...",
      "confidence_score": 7.2,
      "devil_score": 4.1,
      "vertical": "solar_australia",
      "bucket": "explore",
      "created_at": "2026-03-09T07:00:00"
    }
  ]
}
```

```
curl -H "Authorization: Bearer $GATE_API_KEY" http://localhost:5000/pending
```

POST /approve/

Approve an experiment and allocate budget using Kelly Criterion.

Request body (optional):

```
{
  "approved_by": "martin"
}
```

Response 200:

```
{
  "status": "approved",
  "experiment_id": 42,
  "budget_allocated_aud": 72.92
}
```

Response 404: Experiment not found.

Response 400: Experiment not in 'pending' status.

```
curl -X POST http://localhost:5000/approve/42 \
  -H "Authorization: Bearer $GATE_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"approved_by": "martin"}'
```

POST /reject/

Reject a pending or approved experiment.

Request body (optional):

```
{
  "reason": "Budget too tight this week",
  "rejected_by": "martin"
}
```

Response 200:

```
{
  "status": "rejected",
  "experiment_id": 42,
  "reason": "Budget too tight this week"
}
```

```
curl -X POST http://localhost:5000/reject/42 \
  -H "Authorization: Bearer $GATE_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"reason": "Not aligned with current strategy", "rejected_by": "martin"}'
```

GET /dashboard

Full swarm status overview. Returns swarm summary, portfolio breakdown, pending/active experiments, recent leads, and circuit breaker history.

Response:

```
{
  "swarm": {
    "active_experiments": 3,
    "pending_approval": 1,
    "budget_used_aud": 120.50,
    "circuit_breaker": "green"
  },
  "portfolio": { ... },
  "pending_experiments": [...],
  "active_experiments": [...],
  "recent_leads": [...],
  "circuit_breaker_history": [...],
  "generated_at": "2026-03-09T08:00:00"
}
```

```
curl -H "Authorization: Bearer $GATE_API_KEY" http://localhost:5000/dashboard
```

GET /experiments

List recent experiments with optional status filter.

Query params:

PARAM	DEFAULT	DESCRIPTION
`status`	(none)	Filter by status: pending/approved/running/complete/killed/rejected
`limit`	20	Max results (capped at 100)

Response:

```
{
  "experiments": [...],
  "count": 5
}
```

```
# All recent
curl -H "Authorization: Bearer $GATE_API_KEY" http://localhost:5000/experiments

# Running only
curl -H "Authorization: Bearer $GATE_API_KEY" \
  "http://localhost:5000/experiments?status=running&limit=10"
```

GET /costs

API usage and cost breakdown.

Query params:

PARAM	DEFAULT	DESCRIPTION
`days`	7	Look-back period (max 365)
`call_id`	(none)	Cost for a single voice call
`client_id`	(none)	Cost for a specific client

Response (no filters):

```
{
  "summary": {
    "period_days": 7,
    "breakdown": {
      "openai": {"cost_usd": 1.24, "calls": 48},
      "retell": {"cost_usd": 0.84, "calls": 12}
    },
    "total_usd": 2.08,
    "total_aud": 3.22
  },
  "daily": [...],
  "projection": {
    "last_7_days_usd": 2.08,
    "projected_month_usd": 8.91,
    "projected_month_aud": 13.82,
    "margin_note": "At $14 AUD/month costs vs $1,500-2,000 retainer = ~99% margin"
  }
}
```

```
# Full summary (7 days)
curl -H "Authorization: Bearer $GATE_API_KEY" http://localhost:5000/costs

# 30-day view
curl -H "Authorization: Bearer $GATE_API_KEY" "http://localhost:5000/costs?days=30"

# Single call
curl -H "Authorization: Bearer $GATE_API_KEY" \
  "http://localhost:5000/costs?call_id=call_abc123"

# Per client
curl -H "Authorization: Bearer $GATE_API_KEY" \
  "http://localhost:5000/costs?client_id=sunpower_perth&days=30"
```

POST /approve-breaker

Reset a Red circuit breaker. Requires human approval.

Request body:

```
{
  "approved_by": "martin"
}
```

Response 200:

```
{
  "success": true,
  "message": "Circuit breaker reset by martin"
}
```

```
curl -X POST http://localhost:5000/approve-breaker \
  -H "Authorization: Bearer $GATE_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"approved_by": "martin"}'
```

POST /slack/actions

Handles Slack interactive button callbacks (approve/reject experiment buttons posted by the swarm). Validates the request using `SLACK_SIGNING_SECRET` + timestamp. Requests older than 5 minutes are rejected (replay protection).

Request: Form-encoded `payload` field containing JSON (sent automatically by Slack).

Response 200:

```
{
  "status": "ok",
  "result": {"status": "approved", "experiment_id": 42, "budget_allocated_aud": 72.92}
}
```

Response 403: Invalid Slack signature.

This endpoint is called by Slack automatically — not for direct use.

GHL Webhook Server — Port 5001

Receives events from GoHighLevel. All endpoints verify `X-GHL-Signature` HMAC-SHA256 header using `GHL_WEBHOOK_SECRET`. If the secret is not configured, the check is skipped with a warning.

GET /health

```
curl http://localhost:5001/health
```

Response:

```
{
  "status": "ok",
  "service": "ghl-webhooks",
  "timestamp": "2026-03-09T08:00:00"
}
```

POST /webhook/new-lead

New contact created in GHL. Saves to the `leads` table and triggers qualification scoring. If score >= 7 and phone is present, triggers an outbound Retell call.

GHL webhook event: Contact Created

Payload fields read:

FIELD	GHL KEY
name	`full_name`, `first_name` + `last_name`, or `name`
phone	`phone` or `phoneRaw`
email	`email`
suburb	`suburb` or `city`
state	`state`
homeowner_status	`homeowner_status` or `customField.homeowner_status`
monthly_bill	`monthly_bill` or `customField.monthly_bill`
roof_type	`roof_type` or `customField.roof_type`
roof_age	`roof_age` or `customField.roof_age`
pipeline_stage	`pipelineStage` or `pipeline_stage`

Response 200:

```
{
  "status": "processed",
  "lead_id": 7,
  "score": 8.2
}
```

```
# Test with a simulated lead
curl -X POST http://localhost:5001/webhook/new-lead \
  -H "Content-Type: application/json" \
  -d '{
    "full_name": "Jane Smith",
    "phone": "+61412345678",
    "email": "jane@example.com",
    "suburb": "Subiaco",
    "state": "WA",
    "homeowner_status": "owner",
    "monthly_bill": 350,
    "roof_type": "tile",
    "roof_age": 5
  }'
```

POST /webhook/call-complete

Voice AI call finished event from GHL. Updates the lead record with call outcome and timestamps.

GHL webhook event: Conversation Status Changed

Payload fields read: `contactId`, `outcome`, `phone`

Response 200:

```
{
  "status": "processed",
  "contact_id": "abc123",
  "outcome": "booked_assessment"
}
```

POST /webhook/form-submit

Solar quote form submitted via GHL Funnel. Processes richer form data including electricity bill and homeowner status. Saves and qualifies the lead.

Payload fields read: Same as new-lead, plus `homeowner` (maps to homeowner_status), `electricity_bill` or `bill` (maps to monthly_bill), `roof` (maps to roof_type).

Response 200:

```
{
  "status": "processed",
  "lead_id": 8,
  "score": 6.5
}
```

POST /webhook/stage-change

Pipeline stage changed in GHL. Updates `pipeline_stage` on the matching lead record. If the new stage contains "convert", also sets `status = 'converted'`.

GHL webhook event: Opportunity Stage Change

Payload fields read: `contactId` (or `contact_id`), `newStage` (or `stage`)

Response 200:

```
{
  "status": "processed",
  "contact_id": "abc123",
  "new_stage": "Proposal Sent"
}
```

Voice Webhook Server — Port 5002

Receives events from Retell AI. In-memory call context stored in `_call_contexts` dict keyed by `call_id`.

GET /voice/health

```
curl http://localhost:5002/voice/health
```

Response:

```
{"status": "ok", "service": "voice-webhook"}
```

POST /voice/call-started

Retell sends this when a call is initiated. Initialises call context with `client_id`, phones, and empty `lead_data`.

Payload:

```
{
  "call_id": "call_abc123",
  "from_number": "+61412345678",
  "to_number": "+61892345678",
  "agent_id": "agent_xyz"
}
```

Response 200:

```
{"status": "ok"}
```

POST /voice/response

Main LLM endpoint. Retell sends the transcript, the server calls GPT-4o with the full system prompt, and returns the next response. This endpoint is called on every agent turn during a call.

Payload (Retell Custom LLM format):

```
{
  "call_id": "call_abc123",
  "interaction_type": "response_required",
  "response_id": 5,
  "transcript": [
    {"role": "agent", "content": "Hi! Thanks for calling SunPower Perth."},
    {"role": "user", "content": "Hi, I'm interested in solar panels."}
  ]
}
```

Response:

```
{
  "response_id": 5,
  "content": "Great! Let me ask you a few quick questions to see if solar is right for your home.",
  "content_complete": true,
  "end_call": false
}
```

POST /voice/elevenlabs/response

Alternative LLM endpoint used when ElevenLabs Conversational AI is the telephony layer instead of Retell. Accepts the same format as `/voice/response`.

POST /voice/post-call

Retell sends the full call data after hang-up. Triggers 9-step post-call processing: extract, merge, upsert lead, score, update GHL, create task, update call_logs, Slack notification, cold ledger.

Payload:

```
{
  "call_id": "call_abc123",
  "transcript": [...],
  "duration_ms": 180000,
  "recording_url": "https://retell.com/recordings/...",
  "agent_id": "agent_xyz"
}
```

Response 200:

```
{"status": "processed", "call_id": "call_abc123"}
```

Dashboard API — Port 5003

CORS-enabled. Allows requests from `http://localhost:5173` (Vite dev), `http://localhost:4173` (Vite preview), and `FRONTEND_URL` env var. The React swarm board polls these endpoints every 30 seconds.

No authentication by default (internal network service). Add `FRONTEND_URL` in `.env` for production deployments.

GET /api/health

Service health with CRM connection status.

Response:

```
{
  "status": "ok",
  "service": "dashboard-api",
  "crm": {
    "active": "ghl",
    "ghl": true,
    "hubspot": false,
    "salesforce": false
  },
  "timestamp": "2026-03-09T08:00:00"
}
```

```
curl http://localhost:5003/api/health
```

GET /api/crm/status

Which CRM is active and all configured integrations.

Response:

```
{
  "active": "ghl",
  "configured": ["ghl"],
  "detail": {
    "active": "ghl",
    "ghl": true,
    "hubspot": false,
    "salesforce": false
  }
}
```

```
curl http://localhost:5003/api/crm/status
```

GET /api/crm/pipeline

Pipeline stages with contact counts. Served from `crm_cache` table (refreshed every 30 minutes by `crm_sync`). Falls back to a live CRM call if the cache is empty.

Response:

```
{
  "source": "cache",
  "stages": [
    {"name": "New Lead", "count": 12},
    {"name": "Qualified", "count": 5},
    {"name": "Proposal Sent", "count": 3}
  ],
  "cached_at": "2026-03-09T07:30:00"
}
```

```
curl http://localhost:5003/api/crm/pipeline
```

GET /api/crm/contacts

Recent contacts from the cache table.

Query params:

PARAM	DEFAULT	DESCRIPTION
`limit`	20	Max results

Response:

```
{
  "contacts": [...],
  "count": 20
}
```

```
curl "http://localhost:5003/api/crm/contacts?limit=10"
```

GET /api/crm/metrics

Conversion funnel metrics from `crm_cache` (`metrics_summary` key).

Response:

```
{
  "source": "cache",
  "metrics": {
    "total_contacts": 147,
    "new_this_week": 23,
    "pipeline_stages": [...],
    "conversion_rate": 0.12,
    "synced_at": "2026-03-09T07:30:00"
  }
}
```

```
curl http://localhost:5003/api/crm/metrics
```

GET /api/swarm/summary

Hot memory swarm overview: active experiments, pending approvals, budget used, circuit breaker state.

Response:

```
{
  "active_experiments": 3,
  "pending_approval": 1,
  "budget_used_aud": 120.50,
  "circuit_breaker": "green"
}
```

```
curl http://localhost:5003/api/swarm/summary
```

GET /api/swarm/experiments

Recent experiments, optionally filtered by status.

Query params:

PARAM	DEFAULT	DESCRIPTION
`status`	(none)	Filter: pending/approved/running/complete/killed/rejected
`limit`	15	Max results

```
curl "http://localhost:5003/api/swarm/experiments?status=running"
```

GET /api/swarm/leads

Recent leads from the `leads` table. Returns id, name, qualification_score, recommended_action, status, created_at.

Query params:

PARAM	DEFAULT	DESCRIPTION
`limit`	20	Max results

```
curl "http://localhost:5003/api/swarm/leads?limit=50"
```

GET /api/swarm/circuit-breaker

Current circuit breaker state.

Response:

```
{
  "level": "green",
  "halted": false
}
```

```
curl http://localhost:5003/api/swarm/circuit-breaker
```

GET /api/board/state

`board-state.json` merged with live DB counts. Used by the React board's Overview tab.

Response:

```
{
  "...board-state.json contents...",
  "liveStats": {
    "experiments": {
      "pending": 1,
      "approved": 2,
      "running": 1,
      "complete": 8
    },
    "totalLeads": 147,
    "abTestsRunning": 2,
    "generatedAt": "2026-03-09T08:00:00"
  }
}
```

```
curl http://localhost:5003/api/board/state
```

Auth & Admin Blueprints (Port 5003)

Additional routes registered as Flask blueprints on the Dashboard API:

BLUEPRINT	PREFIX	PURPOSE
`api/auth.py`	`/auth/...`	Login, JWT token issue/refresh
`api/users_api.py`	`/users/...`	User management (owner/admin/client roles)
`api/settings_api.py`	`/settings/...`	Runtime config overrides (`app_settings` table)
`api/company_api.py`	`/companies/...`	Solar SME client profiles
`api/apikeys_api.py`	`/apikeys/...`	Embed/webhook API key management

These blueprints use JWT auth (`PyJWT`) and bcrypt password hashing. The `seed_owner()` call on startup creates the initial owner account if none exists.

Swarm Board — React App

Overview

The Swarm Board is a standalone React web application that gives an operator a visual command centre for the Solar Swarm. It runs on `http://localhost:5173` (Vite dev server) or `http://localhost:4173` (Vite preview/production build).

Files:

- `swarm-board/src/App.jsx` — Main component; all UI logic in one file
- `swarm-board/src/components/Confirm.jsx` — Confirmation modal
- `swarm-board/src/components/InfoTip.jsx` — Tooltip component
- `swarm-board/package.json` — React 18.3.1, Vite 5.4.2

Design aesthetic: Dark ops terminal x mission control. Background `#050810`, amber/cyan accent colours, Syne Mono for code elements, DM Sans for body text.

Starting the Board

```
cd swarm-board
npm install
npm run dev
# Open http://localhost:5173
```

For a production build:

```
npm run build
npm run preview   # serves on :4173
```

The Dashboard API (`python main.py` on port 5003) must also be running for the Overview tab to show live data.

Board Tab — Task Kanban

The Board tab is a full Kanban task tracker for managing agent work items. All data persists in browser `localStorage` (no server storage required).

Columns

COLUMN	WIP LIMIT	PURPOSE
BACKLOG	None	Ideas not yet started
QUEUED	8	Ready to work
IN PROGRESS	4	Actively being worked
REVIEW	4	Awaiting review
DONE	None	Completed
BLOCKED	None	Waiting on something external

Seed Tasks

The board ships with 17 pre-loaded tasks (`SEED_TASKS`) that represent the actual project backlog. These load on first visit when `localStorage` is empty. Notable seed tasks include:

- Wire async lead queue to prevent GHL retries (critical, in-progress)
- Add X-API-Key auth to Human Gate API (critical, in-progress)
- Australian solar SME market size deep-dive (high, queued)
- Identify 20 GHL solar companies in Perth + Sydney (high, queued)
- Write 3-touch email follow-up sequence (high, queued)
- DocuSign/PandaDoc e-sign integration (high, blocked — pending provider decision)

Agents

Each task is assigned to one of 13 agents displayed as coloured pills:

AGENT	TIER	ROLE
The General	1	Strategic Command
Research Head	2	Market Intelligence
Content Head	2	Copy & Creative
Analytics Head	2	Performance Analysis
Scout	3	Prospect Hunter
Qualification	3	Lead Scorer
Voice AI	3	Outbound Caller
Proposal	3	Proposal Generator
A/B Evaluator	3	Test Evaluator
Mutation Engine	3	Strategy Evolver
Retrospective	3	Learning Synthesis
Pipeline Proc.	3	Data Processor
Unassigned	—	Not yet assigned

Priorities

PRIORITY	COLOUR
CRITICAL	Red
HIGH	Orange
NORMAL	Blue
LOW	Muted grey

Categories

Lead Pipeline, Experiment, Research, Content, Infrastructure, Client Delivery, Capital, Integration, Bug Fix, Feature.

Creating a Task

Click the `+` button at the top of any column to open the Add Task panel. Required fields: title. Optional: agent, priority, category, tags (comma-separated), description.

Editing a Task

Click any task card to open the detail panel. All fields are editable in-place. Changes save immediately to `localStorage`.

Dragging Tasks

Tasks can be dragged between columns. The column WIP limit is enforced visually (column header turns red when at capacity) but does not block drops.

Deleting a Task

Open the task detail panel and click Delete. A confirmation modal appears before the task is removed.

Filtering

The toolbar above the board provides:

- **Agent filter** — show only tasks assigned to a specific agent
- **Priority filter** — show only tasks of a specific priority
- **Category filter** — show only tasks in a category
- **Search** — full-text search across title, description, and tags

Multiple filters can be active simultaneously.

Overview Tab — Live Metrics

The Overview tab polls the Dashboard API every 30 seconds to display live system state. It requires `python main.py` to be running.

Panels

Swarm Status — Reads from `GET /api/swarm/summary`:

- Active experiments count
- Pending approval count
- Budget used this week (AUD)
- Circuit breaker level (Green / Yellow / Orange / Red)

CRM Status — Reads from `GET /api/crm/status`:

- Active CRM (GHL / HubSpot / Salesforce / none)
- Which integrations are configured

Pipeline — Reads from `GET /api/crm/pipeline`:

- Stage names and contact counts
- Cache age

Recent Leads — Reads from `GET /api/swarm/leads`:

- Last 20 leads with name, score, recommended action, status

Live Stats — Reads from `GET /api/board/state`:

- Experiment counts by status (pending / approved / running / complete)
- Total lead count
- Running A/B tests

Circuit Breaker — Reads from `GET /api/swarm/circuit-breaker`:

- Current level with colour coding
- Halted flag

---

board-state.json

File: `public/board-state.json`

A JSON file in the project's `public/` directory. The `GET /api/board/state` endpoint reads this file and merges it with live DB counts before returning the response to the React board.

The file can be edited manually to add static context (such as current sprint goals or notes) that appears in the Overview tab alongside the live data.

Structure (example):

```
{
  "sprintGoal": "Land first paying client",
  "weeklyBudgetAUD": 500,
  "notes": "Focus on Perth outreach this week"
}
```

The API appends a `liveStats` key to this object before returning it.

---

localStorage Keys

The board stores all task data under these localStorage keys:

KEY	CONTENTS
`swarm_tasks`	JSON array of all task objects
`swarm_filters`	Active filter state

To reset the board to seed tasks, clear `swarm_tasks` from DevTools → Application → Local Storage and reload.

Connecting to an AI Assistant (Optional)

The board has a built-in AI assistant panel that can answer questions about the swarm using the Anthropic API directly from the browser. To enable it:

1. Open the board at `http://localhost:5173`
2. Click the AI assistant icon in the toolbar
3. Enter your Anthropic API key when prompted (stored in localStorage, never sent to the backend)

The key is stored under `anthropic_key` in localStorage. The assistant has read access to the current board state and can help prioritise tasks or answer questions about the architecture.

Note: This uses the Anthropic API directly from the browser. Browser-side API key usage is appropriate for personal/local tools only — do not expose this interface publicly.

Tech Stack

PACKAGE	VERSION	PURPOSE
react	18.3.1	UI framework
@vitejs/plugin-react	4.3.1	Vite React plugin
vite	5.4.2	Build tool and dev server

No additional UI libraries — all components are built inline in `App.jsx` using inline styles with the colour constants defined in the `C` object at the top of the file.